INDIVIDUAL PROJECT REPORT

Review 3 — Final Prototype, Report & Viva (9/10/2026)

AI-Driven Edge–Cloud Network for Cyclone Prediction and Resilient Ocean Telemetry

Student	Register No.	Team No.	My role in the team
Anvay Sharma	24BCE0824	_____	Edge AI (satellite feature extraction and ML early-warning classifier), edge gateway, LSTM implementation, evaluation harness, trained models and run logs

Team members: Laksh Ahuja (24BDS0212), Anvay Sharma (24BCE0824), Sreehari (24BDS0417).
Source code: https://github.com/laksh-ahuja06/AI-DRIVEN-EDGE-CLOUD-NETWORK

Abstract

Our project is a cyclone-monitoring network in which sensing nodes and a satellite ground station report to an edge gateway that forwards data by importance to a central prediction server. My work is the intelligence at the edge and the evidence behind it. I implemented the edge-AI module, which reduces a 2.1 MB four-band satellite frame to a roughly 1 KB feature record using Dvorak-inspired cues, and a logistic-regression classifier that predicts a threshold breach within the next ten readings from the trend of a node's data; the edge gateway that validates, classifies and forwards every packet; a long short-term memory network written from scratch in numpy with backpropagation through time and Adam; and the evaluation harness that produced all reported numbers. The classifier warned on average 16.5 readings before the fixed thresholds were crossed, raising recall from 0.70 to 0.99 at a precision of 0.72. The LSTM reached a validation error 66 % below a no-change forecast, and its gradients matched numerical estimates to a relative error of 3×10^{-7}. Edge feature extraction saved 99.95 % of satellite bandwidth.

Keywords: edge AI, logistic regression, LSTM, Dvorak technique, satellite imagery, early warning, evaluation

1. Introduction

1.1 Problem statement

A cyclone-warning network must decide, at the edge and within milliseconds, which data is urgent; if it can only react once a dangerous value has been measured, and if it ships every multi-megabyte satellite image unprocessed, warnings arrive late and the backbone chokes.

1.2 Why it matters and who is affected

Tropical cyclones are among the deadliest and costliest weather extremes recorded by the WMO [1], and India's east coast is repeatedly exposed [2]. Fishing communities, coastal residents and the authorities that order evacuations need warnings as early as possible. A threshold-only network gives **zero** readings of lead time by definition, and a single raw satellite frame (2.1 MB) would occupy a 9.6 kbps link for about half an hour — the consequence is late or missing alerts.

1.3 Objectives of my part

- Extract compact, explainable storm features from satellite frames at the ground station.
- Flag danger from the trend of the data, before thresholds are crossed, without hiding threshold breaches.
- Build the gateway that validates, classifies and hands every packet to the scheduler.
- Implement and verify the LSTM used for reconstruction, and measure every claim reproducibly.

2. Literature Review

Table 1: Literature most relevant to my modules

Author / Year	Method used	Result	Limitation for our problem
Dvorak, 1975 [3]	Expert pattern analysis of satellite cloud features	Operational intensity estimates worldwide	Subjective, analyst-dependent
Velden et al., 2006 [4]	Review of 30 years of the Dvorak technique	Explains strengths and error sources	Needs trained analysts and full images
Olander & Velden, 2007 [5]	Advanced Dvorak Technique: objective IR scheme	Automated intensity from geostationary IR	Runs centrally on full-resolution imagery
Pradhan et al., 2018 [6]	Deep CNN on IR satellite images	Learned intensity estimation with good accuracy	GPU-heavy, needs large labelled datasets
Hochreiter & Schmidhuber, 1997 [7]	LSTM recurrent network	Learns long-range temporal dependencies	Generic model; no telemetry pipeline
Che et al., 2018 [8]	GRU-D: RNN with missing-value decay	Better accuracy on series with gaps	Framework-heavy; offline clinical data
Kingma & Ba, 2015 [9]	Adam adaptive optimiser	Fast, robust training	Optimiser only
Shi et al., 2016 [10]	Edge computing vision	Motivates processing near sources	No concrete edge-AI evaluation

Gap my part fills: objective Dvorak-style schemes and CNNs run centrally on full images; none sends only the features over a constrained link. Existing QoS classifiers use the current value only. My modules move a lightweight, explainable image analysis to the ground station and add a trend-based classifier at the gateway that runs on a Raspberry-class CPU with only numpy.

3. Methodology

FIG. 1

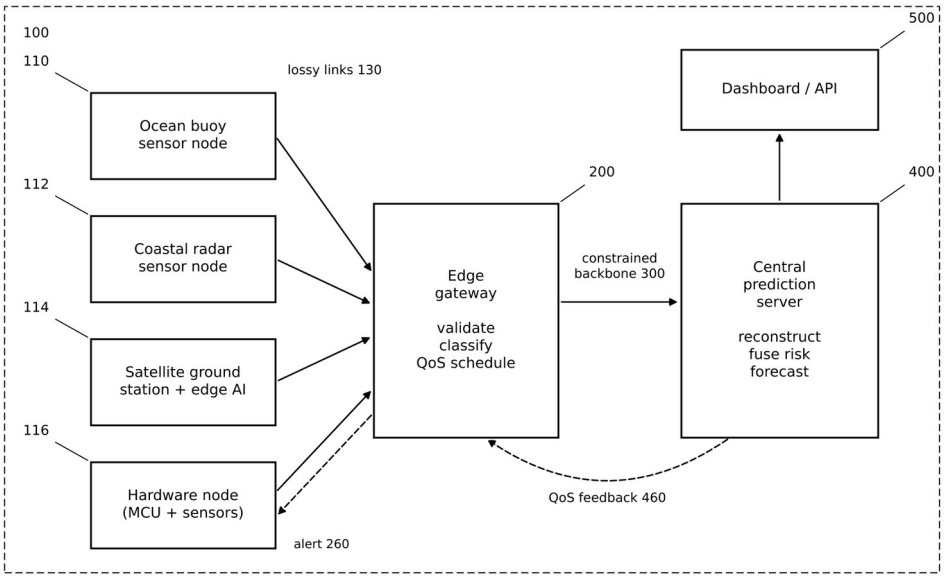


Figure 1: System architecture; my modules: edge AI (in 114 and 224) and the edge gateway 200

3.1 Edge gateway pipeline (edge_gateway.py)

Input → process → output (Figure 2): a datagram is received → parsed and validated (node ID, sequence number, numeric fields in physical ranges) → its node history buffer (last 6 readings) is updated → the rule classifier and the ML classifier both run → hybrid decision (CRITICAL if either says so) → the packet is annotated (class, ML probability, gateway timestamp) and handed to the QoS scheduler → forwarded. The gateway also accepts alert feedback from the server and notifies hardware nodes every 2 s.

FIG. 2

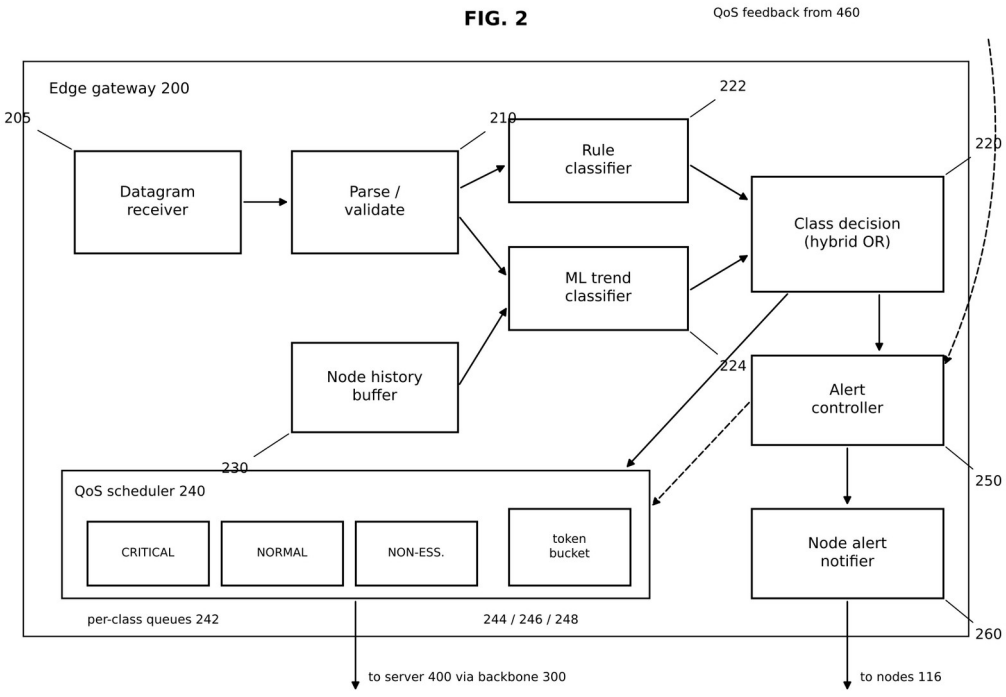


Figure 2: Edge gateway functional blocks (205–260)

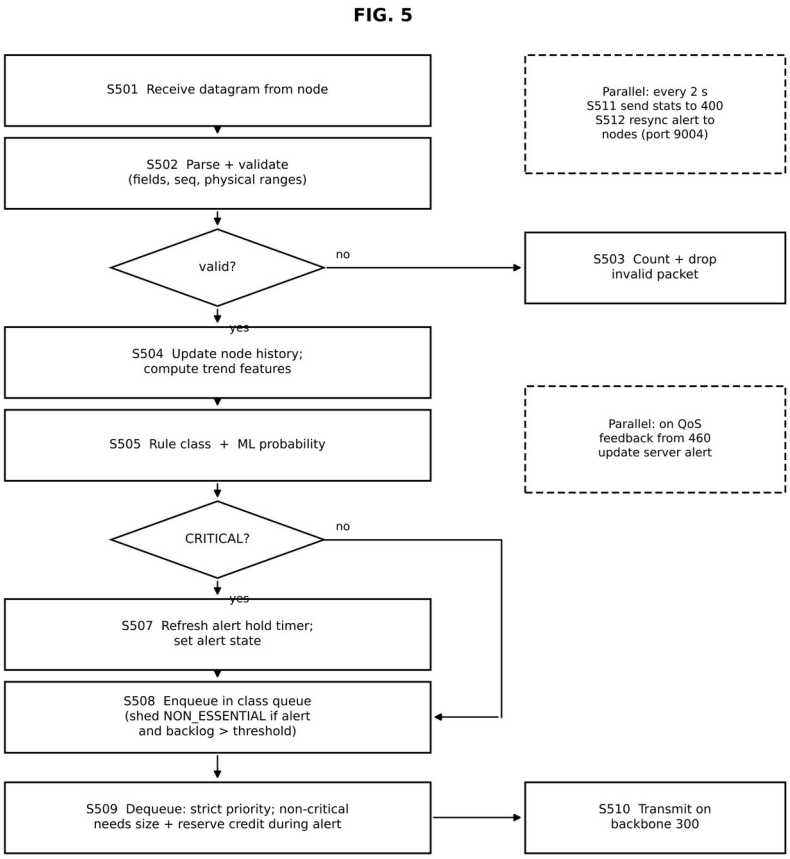


Figure 3: Gateway method flowchart (S501–S512)

3.2 ML early-warning classifier (edge_ai.QoSClassifier)

Seven features are computed per reading: current wind and pressure, the per-reading trend of wind, pressure and temperature over the history window, and the distance of wind and pressure to their thresholds. A logistic regression with feature standardisation and class weighting outputs P(breach within 10 readings); labels come from 240 simulated scenarios in which the future is known.

3.3 Satellite feature extraction (edge_ai.extract_features)

From the infrared band: cold-cloud fraction (< 220 K), coldest cloud top, coldness-weighted storm centre, axisymmetry over eight angular sectors and eye contrast (warm centre minus eyewall). These mirror the cues of the Dvorak technique [3, 4]. The risk score is 40·cover·depth + 30·symmetry·cover + 30·eye, flagged at 60 (Figure 4). This part is classical, rule-based image analysis; the learned components are the classifier and the LSTM.

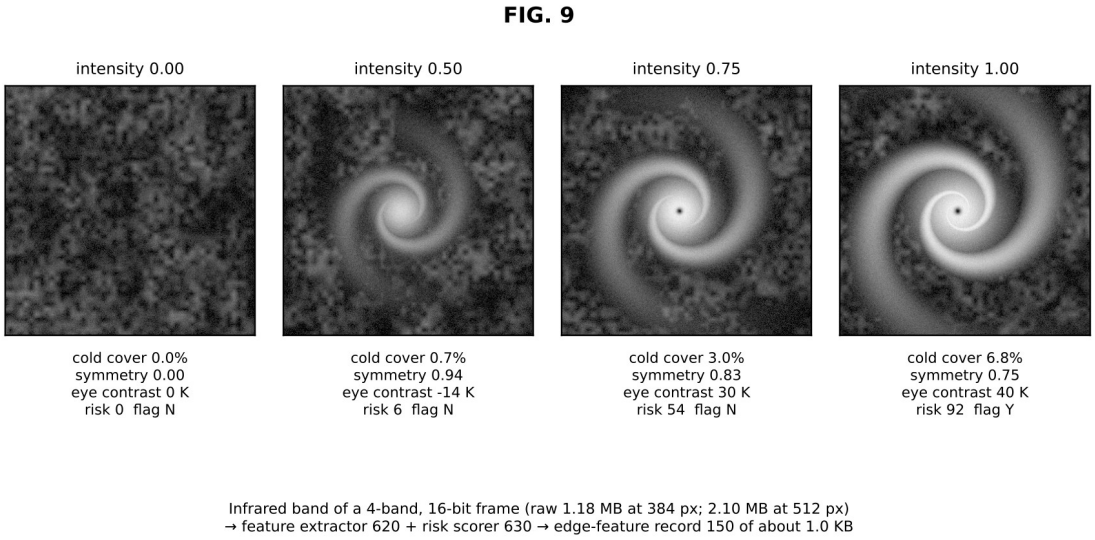


Figure 4: Features and risk extracted from frames of increasing storm intensity

3.4 LSTM (lstm.py)

A standard LSTM cell [7] with forget-gate bias 1, a linear read-out, mean-squared-error loss, backpropagation through time, gradient clipping at norm 5 and the Adam optimiser [9]. Input: 12 steps × 6 features (normalised levels and first differences); output: the next change in wind, temperature and pressure. Hidden size 32; about 5,100 parameters.

3.5 Justification of design choices

Table 2: Design choices for my modules

Decision	Alternatives considered	Chosen	Reason
Early-warning model	Threshold rule only; random forest; small neural network	Logistic regression + rule (hybrid)	Explainable weights, trains in < 1 s, runs on a Pi; rule guarantees no missed breach
Satellite analysis	Send raw frames; zlib compression; CNN intensity model	Dvorak-style features at the edge	99.95 % saving vs 17–22 % for zlib; no GPU or labelled real imagery needed
Sequence model	Linear interpolation; GRU; LSTM in PyTorch	LSTM in numpy	Captures dynamics; no 550 MB framework; verifiable with a gradient check
Optimiser	SGD; RMSprop; Adam	Adam with clipping	Stable, fast convergence on small batches
Evaluation	Manual demo runs; live measurements only	Scripted harness, held-out scenarios, same scheduler code with a simulated clock	Reproducible numbers; fair baselines

4. Implementation

Table 3: Files I implemented

File	Lines	Key functions / classes	What I implemented
edge_ai.py	208	extract_features, satellite_risk, trend_features, NodeHistory, QoSClassifier.fit/predict	Image features, risk score, thumbnail, logistic regression (numpy)
edge_gateway.py	237	Gateway.handle_packet, classify, forwarder, control_listener, stats_reporter	Validation, hybrid classification, alert logic, feedback, node alerts, statistics

File	Lines	Key functions / classes	What I implemented
lstm.py	151	LSTMRegressor.forward, loss_and_grads, adam_step, fit, save/load	Complete LSTM with BPTT and Adam, no external DL framework
evaluate.py	396	eval_recovery, simulate_qos, eval_classifier, eval_bandwidth, write_report	Reproducible experiments → results/REPORT.md + plots
models/	2 files	lstm_recon.npz, qos_classifier.npz	Trained weights shipped with the code
logs/	9 files	per-component logs + truth CSVs	Run evidence used in all reports

The backward pass of the LSTM is the part most likely to hide a bug, so I verified it against a numerical gradient (central differences, $\varepsilon = 10^{-6}$). The maximum relative error was 2.9×10^{-7}, and the check is kept as an automated test.

```
for z, i, f, o, g, c_prev, tc in reversed(cache):     # backprop through time
    do = dh * tc
    dc = dc + dh * o * (1 - tc ** 2)
    da = concat([dc*g*i*(1-i), dc*c_prev*f*(1-f), do*o*(1-o), dc*i*(1-g**2)])
    grads['W'] += z.T @ da;  dh = (da @ W.T)[:, n_in:];  dc = dc * f
```

5. Testing and Results

Table 4: Benchmarks for my modules

Requirement (benchmark)	Test setup	Result	Met?
Warn before threshold breach	60 held-out scenarios × 190 readings	Mean lead 16.5 readings (rule: 0)	Yes
Recall of upcoming breaches ≥ 0.95	Same	0.99 hybrid (rule 0.70)	Yes
Precision ≥ 0.70	Same	0.72	Yes (narrowly)
LSTM better than no-change forecast	1,520 validation windows	MSE 0.089 vs 0.265 (−66 %)	Yes
Gradient correctness	Numerical check, all weight matrices	rel. error 2.9×10^{-7}	Yes
Satellite bandwidth reduction ≥ 95 %	9 frames, 3 sizes	99.80–99.99 %	Yes
Edge inference < 100 ms per 512^2 frame	Laptop CPU	≈13 ms	Yes
Gateway rejects corrupt packets	Malformed JSON, out-of-range values	Counted and dropped (unit test)	Yes

Table 5: Classifier results on held-out scenarios

Policy	Precision	Recall	F1	Mean lead (readings)	Calm scenarios with false alarm
rule	1.0	0.695	0.82	0.0	0 of 9
ml	0.724	0.988	0.835	16.45	3 of 9
hybrid	0.724	0.99	0.837	16.45	3 of 9

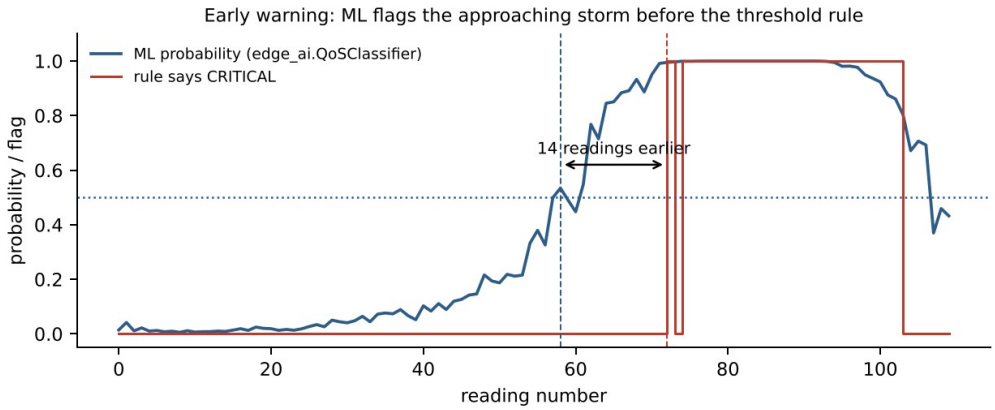


Figure 5: Example scenario: ML probability crosses 0.5 fourteen readings before the rule fires

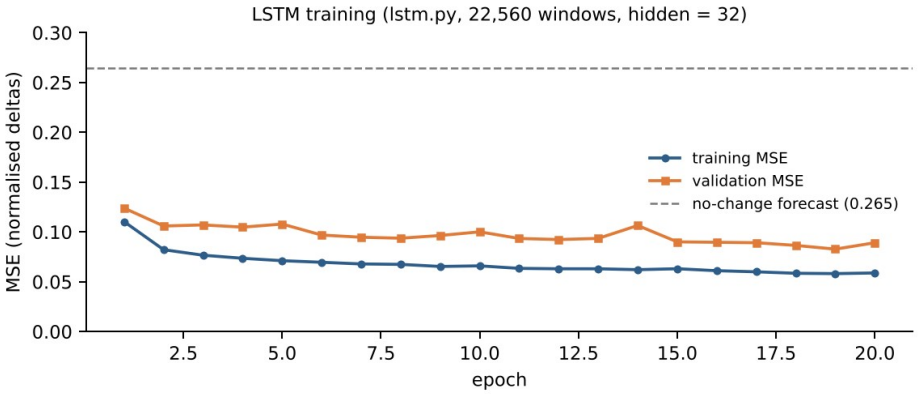


Figure 6: LSTM training and validation error (from train_log.txt)

In the live run the gateway forwarded 52 critical, 170 normal and 39 non-essential packets with no queue drops, shed 3 housekeeping packets during the alert, and flagged 29 packets as critical through the ML classifier before the rule would have (Figure 7).

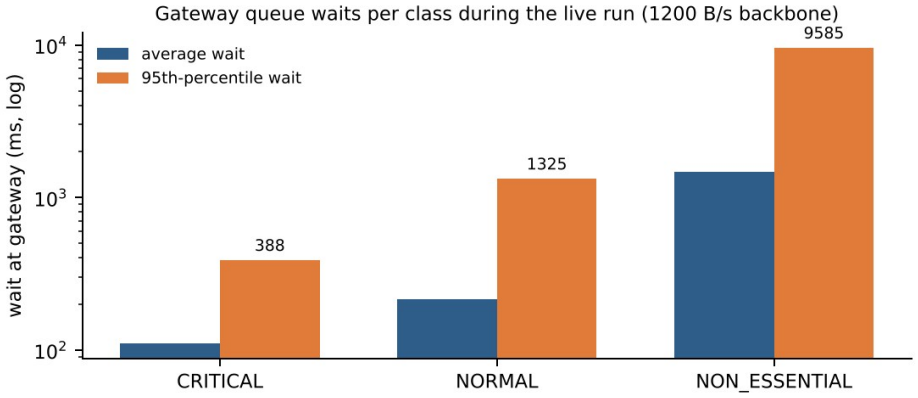


Figure 7: Gateway waits per class in the live run (from my gateway statistics)

5.1 Failures found and fixed

Table 6: Failures and fixes in my modules

What failed	Diagnosis	Change	Retest result
PyTorch could not be used on target hardware	≈550 MB download, too heavy for a Pi gateway	Wrote the LSTM in numpy with BPTT and Adam	Trains in 19 s; gradient check passes
LSTM gap-repair looked worse than	Evaluation fed the model a history with earlier gaps	History now contains the model's own earlier	Fair comparison; honest result: equal for wind, better

What failed	Diagnosis	Change	Retest result
linear for wind at 10 % loss	collapsed — not what the server does	estimates	for pressure
Satellite risk stayed near 0 for strong storms	Synthetic cloud tops never reached the 220 K cold threshold, so cold-cloud cover was ≈0	Re-calibrated cover normalisation with my teammate's imager fix	Risk rises 0 → 6 → 48 → 70 → 93 with intensity
Classifier raised false alarms	Trend features fire on calm-day gusts	Kept hybrid policy, reported the trade-off instead of tuning on test data	3 of 9 calm scenarios; documented as a limitation

6. Live Demonstration

1. `python train_models.py` — show training output (≈20 s) and the saved models.
2. `python run_demo.py` — open the dashboard; point to the ML probability column and the satellite panel.
3. Open `logs/gateway.log` — show packets classified CRITICAL early and the 'alert ACTIVE' line.
4. `python evaluate.py` — regenerate `results/REPORT.md` and walk through the classifier and bandwidth tables.
5. `python -m pytest -q` — 28 passed, including the LSTM gradient test.

6.1 Review 3 Video Plan (6 minutes)

Time	Content
0:00–2:00	Live demo: gateway log + dashboard ML column during a run; satellite panel
2:00–4:00	Test results: classifier lead time, LSTM training curve and gradient check, fixes
4:00–5:00	Comparison with Dvorak / ADT / CNN / GRU-D (Section 8)
5:00–6:00	My contribution (edge AI, gateway, LSTM, evaluation) and next steps

The video starts with my name, register number, team number and project title, as required.

7. Feasibility (Five Dimensions)

Table 7: Feasibility with emphasis on my modules

Dimension	Evidence
Technical	All benchmarks in the Section 5 benchmark table met; models train in about 20 s on a laptop CPU; inference needs only numpy.
Economic	No paid software or GPU; edge AI runs on existing hardware. Sending ≈1 KB instead of 2.1 MB per frame cuts satellite airtime cost by ≈99.95 %.
Social	Earlier warnings directly benefit coastal residents; feedback from reviewers to be recorded: _____
Environmental	No GPU training (low energy); smaller transmissions save radio energy at remote stations.
Operational / Safety	Classifier is explainable (7 named features); rule kept as a safety floor so a threshold breach is never missed; gateway rejects malformed data.

8. Comparison with Literature

Table 8: Comparison with five papers from the Review 1 survey

Parameter	Existing work	Our project	Improved?
Where image analysis	Centrally on full images (ADT) [5]	At the ground station; only ≈1 KB sent	Yes

Parameter	Existing work	Our project	Improved?
runs			
Objectivity	Analyst-dependent (Dvorak) [3]	Fully automatic, explainable features	Yes
Intensity accuracy	CNN validated on real storms [6]	Not validated on real imagery	No
Missing-data model	GRU-D learns decay over gaps [8]	LSTM forecast anchored to next reading	Comparable, simpler
Compute needed	Deep-learning frameworks / GPU [6, 8]	numpy only, CPU, ≈5.1 k parameters	Yes

9. Scalability and Next Steps

- Run the feature extractor on real INSAT-3D/3DR infrared tiles from MOSDAC and recalibrate the thresholds.
- Replace logistic regression with gradient-boosted trees or a small temporal model, keeping the rule as a floor.
- Track model versions and retrain automatically from archived telemetry; quantise the LSTM for microcontrollers.
- Multiple gateways: share node histories so a node moving between gateways keeps its trend features.

10. Progress since Review 2

Table 9: What changed in my modules between reviews

Area	At Review 2 (14/9)	Now (Review 3)
Classification	Threshold rule only (2 classes)	Rule + logistic-regression trend classifier (hybrid), 3 classes, validation
Edge AI	Not started	Satellite feature extraction, risk score, thumbnail; 99.95 % bandwidth saving
Learning / evidence	No model; 12-second smoke test	numpy LSTM with BPTT + Adam; evaluation harness with 4 experiments; trained models

Overall completion: software 100 % of planned scope (all modules running and tested); validation on real buoy data not yet started.

11. My Contribution

Table 10: Division of work

Tasks I did	Tasks done by teammates
Edge AI (edge_ai.py): satellite feature extraction, risk score, ML trend classifier	Laksh Ahuja: Central prediction server; Live dashboard; QoS scheduler
Edge gateway (edge_gateway.py): validation, hybrid classification, alerts, feedback, node notifications	Sreehari: Hardware: ESP32 buoy firmware, LoRa buoy/receiver sketches, serial bridge, hardware guide and BOM; Lost-telemetry reconstruction; Satellite ground station
LSTM from scratch (lstm.py) with BPTT and Adam	
Evaluation harness (evaluate.py) → results/REPORT.md and plots	
Trained models (models/) and run logs (logs/)	

Evidence of my work:

- Code: the files listed in my Implementation section (https://github.com/laksh-ahuja06/AI-DRIVEN-EDGE-CLOUD-NETWORK).
- Automated tests covering my modules, all passing (Table below; full output in `docs/reports/evidence/pytest_output.txt`).
- Run logs from the live demonstration: logs/gateway.log (classification, 'alert ACTIVE', reserve lines), results/results.json, train_log.txt (excerpts in Appendix B).
- Figures in this report generated from those logs by `docs/reports/make_figures.py`.

Table 11: Tests covering my modules

Automated test (tests/test_pipeline.py)	What it checks	Result
test_satellite_features_track_intensity_and_compress	Calm frame risk < 10, mature storm > 60 and flagged; record < 1 % of frame	PASSED
test_qos_classifier_learns_simple_rule	Logistic regression reaches > 95 % accuracy on a separable rule	PASSED
test_lstm_gradients_match_numerical	Backpropagation-through-time gradients equal numerical gradients	PASSED
test_lstm_learns_a_sine_wave	LSTM learns a sine wave to MSE < 0.01	PASSED
test_gateway_classifies_and_drops_invalid	Malformed / out-of-range packets dropped and counted; wind 80 km/h → CRITICAL + alert	PASSED
test_gateway_ml_policy_flags_early	Falling pressure / rising wind flagged CRITICAL before thresholds are crossed	PASSED

> *[Insert here: screenshot of my own commits / work session and a photo of me testing — required by the rubric. Shared files (common.py, mininet_topology.py) were developed jointly.]*

12. Viva Preparation

Q: Why logistic regression and not a deep model? Small, explainable, trains in seconds, runs on a Pi; the trend features carry most of the signal.

Q: Why keep the rule in the hybrid policy? So a real threshold breach is never missed even if the ML is wrong.

Q: Is the image analysis machine learning? No — it is Dvorak-inspired classical image processing; the classifier and the LSTM are the learned parts.

Q: How did you know your LSTM gradients were right? Numerical gradient check, relative error 2.9×10^{-7}.

Q: What if the classifier threshold changes from 0.5? Lower → earlier warnings, more false alarms; higher → fewer false alarms, less lead.

13. Conclusion

My modules give the network judgement at the edge: satellite frames shrink to the features that matter, and the gateway recognises an approaching storm from its trend well before thresholds are crossed. The LSTM and the evaluation harness make the reconstruction and every number in our reports reproducible. The key limitations — synthetic imagery and some false alarms — are stated openly and define the next steps.

References

[1] World Meteorological Organization, *WMO Atlas of Mortality and Economic Losses from Weather,*

Climate and Water Extremes (1970–2019), WMO-No. 1267, Geneva, 2021.

[2] India Meteorological Department, "Cyclone eAtlas" and Regional Specialised Meteorological Centre (Tropical Cyclones) reports, New Delhi. [Online]. Available: https://rsmcnewdelhi.imd.gov.in

[3] V. F. Dvorak, "Tropical cyclone intensity analysis and forecasting from satellite imagery," *Monthly Weather Review*, vol. 103, no. 5, pp. 420–430, 1975.

[4] C. Velden *et al.*, "The Dvorak tropical cyclone intensity estimation technique: A satellite-based method that has endured for over 30 years," *Bull. Amer. Meteor. Soc.*, vol. 87, no. 9, pp. 1195–1210, 2006.

[5] T. L. Olander and C. S. Velden, "The Advanced Dvorak Technique: Continued development of an objective scheme to estimate tropical cyclone intensity using geostationary infrared satellite imagery," *Weather and Forecasting*, vol. 22, no. 2, pp. 287–298, 2007.

[6] R. Pradhan, R. S. Aygun, M. Maskey, R. Ramachandran, and D. J. Cecil, "Tropical cyclone intensity estimation using a deep convolutional neural network," *IEEE Trans. Image Processing*, vol. 27, no. 2, pp. 692–702, 2018.

[7] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[8] Z. Che, S. Purushotham, K. Cho, D. Sontag, and Y. Liu, "Recurrent neural networks for multivariate time series with missing values," *Scientific Reports*, vol. 8, art. 6085, 2018.

[9] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.

[10] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[11] S. Blake, D. Black, M. Carlson, E. Davies, Z. Wang, and W. Weiss, "An architecture for differentiated services," IETF RFC 2475, 1998.

Appendix A — How to Run My Modules

Command	Purpose
`python edge_gateway.py --policy hybrid`	Gateway with rule + ML (also: rule, ml)
`python edge_gateway.py --scheduler fifo --backbone-bps 1000`	Gateway baseline, tight link
`python train_models.py`	Retrains LSTM and classifier (≈20 s)
`python evaluate.py`	Reproduces results/REPORT.md and plots
`curl 127.0.0.1:8000/gateway`	Live gateway QoS statistics

Appendix B — Evidence from the Live Run (12/10/2026)

Unedited excerpts from the log files produced by `python run_demo.py --duration 75`:

```
# logs/gateway.log
[gateway] listening for sensors on 0.0.0.0:9001 (policy=hybrid, scheduler=priority, backbone=1200
B/s)
[gateway] cyclone alert ACTIVE (local critical traffic) -> critical bandwidth reserved, non-
essential shed
[gateway] NORMAL        BUOY_07 seq=41 forwarded after 1133 ms  (alert: critical reserve held)
[gateway] CRITICAL      SAT_GS_01 seq=14 forwarded after 969 ms
[gateway] CRITICAL      SAT_GS_01 seq=15 forwarded after 286 ms
[gateway] CRITICAL      SAT_GS_01 seq=16 forwarded after 397 ms
[gateway] CRITICAL      SAT_GS_01 seq=17 forwarded after 383 ms
[gateway] CRITICAL      BUOY_07 seq=52 forwarded after 0 ms
[gateway] cyclone alert ACTIVE (local critical traffic) -> critical bandwidth reserved, non-
essential shed
[gateway] NON_ESSENTIAL BUOY_07 seq=13 shed (alert active)
[gateway] NON_ESSENTIAL BUOY_12 seq=13 shed (alert active)
# train_log.txt
LSTM: 22560 training windows, 1520 validation windows, hidden=32
  epoch   1/20  train mse=0.11032  val mse=0.12391
  epoch  20/20  train mse=0.05882  val mse=0.08914
LSTM trained in 19s; val mse=0.0891 vs 0.2647 for a no-change forecast
QoS classifier: 45600 samples (14.4% positive); val precision=0.732 recall=0.983
```